Leitura — Dívida técnica (Sprint 1, aula de 07/10)

Guia de apoio para a aula de 07/10. Nas aulas anteriores da Sprint 1, a pergunta era "como o trabalho flui, e onde ele trava". Esta leitura trata de uma das causas mais comuns de o trabalho travar com o tempo: o custo acumulado de decisões e atalhos no código, nos testes e no processo. O nome dado a esse custo é dívida técnica.

Os slides da aula são `slides/processos-slides-07.md`. A leitura segue a mesma ordem e acrescenta detalhes, exemplos e exercícios.

Como ler: os capítulos 1 a 3 explicam o conceito; o capítulo 4 mostra as consequências; os capítulos 5 a 7 tratam do que fazer; o capítulo 8 leva o assunto para o projeto do grupo.

Capítulos:

1. A metáfora
2. Principal e juros
3. Tipos de dívida
4. O que acontece quando a dívida cresce
5. Evitar a dívida que não traz benefício
6. Tornar a dívida visível
7. Pagar
8. No projeto do grupo
9. Exercícios e dúvidas frequentes

1. A metáfora

O termo vem de um relato de Ward Cunningham, de 1992, sobre um sistema de gestão de carteiras de investimento. A equipe entregava versões cedo, para aprender com o uso, e precisava explicar aos gestores por que depois voltava ao código para reescrevê-lo. Cunningham usou uma comparação que o público do setor financeiro entendia:

> "Entregar um código pela primeira vez é como contrair uma dívida. Uma pequena dívida acelera o desenvolvimento, desde que seja paga logo, com uma reescrita. [...] O perigo aparece quando a dívida não é paga. Cada minuto gasto em código que não está bem certo conta como juros sobre essa dívida. Organizações inteiras de engenharia podem ficar paralisadas sob o peso da dívida de uma implementação não consolidada."
>
> Ward Cunningham, 1992 (tradução livre)

Dois pontos desse texto costumam se perder quando o termo é usado no dia a dia:

- A dívida, na origem, é uma decisão que ajuda. Entregar cedo permite aprender com o uso real, e esse aprendizado vale o custo de reescrever depois.
- A condição vem junto. O código precisa ser reescrito à medida que a equipe entende melhor o problema. O que Cunningham aponta como perigo é a falta do pagamento.

A metáfora serve, antes de tudo, para conversar com quem não programa. "O código está ruim" não diz nada a quem decide prioridades. "Cada funcionalidade nova neste módulo custa dois dias a mais do que deveria" é uma informação que entra numa decisão.

📖 Ref. Ward Cunningham — The WyCash Portfolio Management System: https://c2.com/doc/oopsla92.html

2. Principal e juros

A comparação com uma dívida financeira tem duas partes:

Termo	Na dívida financeira	No software
Principal	o valor emprestado	o trabalho necessário para deixar o código como deveria estar
Juros	o que se paga a mais enquanto a dívida existe	o esforço a mais em cada mudança feita sobre esse código

Martin Fowler dá um exemplo com números. Uma equipe precisa acrescentar uma funcionalidade a um módulo com estrutura confusa:

```
Funcionalidade nova num módulo bem estruturado ........ 4 dias
A mesma funcionalidade no módulo confuso .............. 6 dias   → 2 dias de juros
Reorganizar o módulo .................................. 5 dias   → o principal
```

A conta mostra por que a decisão depende do que vem pela frente:

- Se essa é a única mudança prevista no módulo, pagar o principal (5 dias) custa mais do que os juros (2 dias). Conviver com a dívida é a escolha razoável.
- Se há três funcionalidades previstas no mesmo módulo, os juros somam 6 dias. Reorganizar antes custa 5 dias e deixa as três mais baratas.
- Se o módulo não vai mais ser alterado, não há juros. Código confuso e estável não cobra nada de ninguém.

Daí sai uma regra prática: o que determina a prioridade de uma dívida é a frequência com que o código é alterado, mais do que o quanto ele está ruim.

Os juros nem sempre aparecem como tempo. Podem aparecer como risco (uma mudança que quebra outra parte do sistema), como defeitos que chegam ao usuário ou como dependência de uma pessoa (só um integrante consegue mexer naquele trecho).

📖 Ref. Martin Fowler — Technical Debt: https://martinfowler.com/bliki/TechnicalDebt.html

3. Tipos de dívida

3.1 Como a dívida aparece

Hoje o termo é usado para vários problemas diferentes. Os mais comuns:

Sintoma	Exemplo num projeto da disciplina
Projeto do código que fazia sentido e deixou de fazer	a estrutura criada na Sprint 0 não comporta as telas ou rotas novas
Defeitos conhecidos e não corrigidos	"esse erro acontece às vezes, depois a gente vê"
Cobertura de testes insuficiente	regra de negócio sem nenhum teste
Teste manual onde caberia teste automatizado	conferir tudo à mão antes de cada entrega
Integração e implantação manuais	só um integrante sabe montar e publicar o projeto
Conhecimento concentrado	só um integrante entende um módulo
Documentação desatualizada	o `README.md` descreve um passo que não funciona mais

Nem tudo é código. Uma decisão não registrada e um processo que ninguém segue também cobram juros: toda vez que alguém precisa perguntar, adivinhar ou refazer.

3.2 Três tipos, pela origem

Kenneth Rubin separa a dívida em três tipos, conforme a origem. A separação é útil porque a resposta a cada um é diferente.

Tipo	Como surge	O que fazer
Ingênua (ou não intencional)	falta de prática, de testes ou de cuidado com o projeto do código	evitar, com boas práticas de engenharia e revisão
Inevitável	o entendimento do produto evolui; uma biblioteca externa muda	aceitar e pagar aos poucos, conforme se aprende
Estratégica	decisão consciente de tomar um atalho para cumprir um objetivo de curto prazo	registrar a decisão e o plano de pagamento

A dívida ingênua é a única que não traz benefício em troca. As outras duas fazem parte de desenvolver software.

A dívida inevitável existe porque ninguém prevê com exatidão como um produto vai evoluir. O bom desenho de um sistema aparece durante a construção, com o que a equipe aprende ao entregar e observar o uso. Um exemplo sem culpa de ninguém: o grupo usa uma biblioteca externa, e uma versão nova muda a forma de chamá-la. O código que estava correto passa a precisar de ajuste.

A dívida estratégica é uma ferramenta. Uma equipe pode decidir entregar com um atalho para cumprir um prazo que importa, como uma demonstração ou uma data de mercado. O que a distingue da ingênua é a consciência: a equipe sabe o que está devendo, por quê, e quando pretende pagar.

📖 Ref. RUBIN, Kenneth S. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley, 2012. Capítulo 8.

3.3 O quadrante: prudente ou imprudente, deliberada ou não

Martin Fowler propõe outra classificação, com duas perguntas. A equipe sabia que estava contraindo a dívida? A decisão foi sensata?

	Deliberada	**Não deliberada**
Imprudente	"Não temos tempo para pensar no projeto do código"	"O que é separação em camadas?"
Prudente	"Precisamos entregar agora e lidar com as consequências depois"	"Agora sabemos como deveríamos ter feito"

- Prudente e deliberada: o ganho de entregar antes compensa os juros, e a equipe fez essa conta. É a dívida estratégica.
- Prudente e não deliberada: acontece com equipes boas. Só depois de construir é que se descobre qual seria o melhor desenho. É a dívida inevitável.
- Imprudente e não deliberada: a equipe não conhece as práticas que evitariam o problema. É a dívida ingênua.
- Imprudente e deliberada: a equipe conhece as boas práticas e decide ignorá-las por pressa. Costuma atrasar mais do que adianta, porque os juros aparecem em poucas semanas.

Para Fowler, a distinção que mais importa é entre prudente e imprudente. Discutir se código desorganizado "merece" ser chamado de dívida ajuda menos do que perguntar se a decisão que o produziu foi sensata.

📖 Ref. Martin Fowler — Technical Debt Quadrant: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

4. O que acontece quando a dívida cresce

Consequência	O que a equipe percebe
Crescimento não linear	a partir de certo ponto, mudanças pequenas passam a ter resultado incerto
Entregas mais lentas	o atalho de hoje reduz a velocidade de amanhã; o tempo entre entregas aumenta
Mais defeitos	o código fica complexo, e corrigir falhas toma o tempo das funcionalidades
Estimativas ruins	ninguém consegue prever quanto uma mudança leva
Perda de confiança	a equipe deixa de cumprir o que combina, e os clientes deixam de contar com ela
Produto parado	sem novidades nem correções, o produto perde usuários

O crescimento não linear é a característica mais importante. A dívida não aumenta um pouco a cada semana, de forma previsível: ela se acumula sem efeito aparente e, a partir de um ponto, cada mudança passa a ter consequências difíceis de antecipar. É um risco para o negócio, e não só um incômodo para quem programa.

O efeito sobre o prazo contraria a intuição. O atalho é tomado para entregar mais rápido, e funciona na primeira entrega. Nas seguintes, a velocidade cai, e o tempo entre entregas aumenta.

Quanto tempo leva para o custo aparecer? Fowler defende que é pouco: equipes que descuidam da qualidade interna do código sentem a perda de velocidade em semanas, não em meses. Ele mesmo apresenta isso como uma hipótese, apoiada na experiência de quem desenvolve, e não como um resultado medido. No mesmo artigo, ele cita um estudo de Tornhill e Borg segundo o qual resolver uma tarefa em código de baixa qualidade levou mais que o dobro do tempo, com densidade de defeitos quinze vezes maior.

Nas métricas que os grupos já coletam, a dívida costuma aparecer assim:

- o *cycle time* sobe sem que as tarefas tenham ficado maiores;
- as falhas de CI aumentam;
- cresce o retrabalho: cartões que voltam de "Em revisão" para "Em progresso", ou PRs que corrigem PRs anteriores.

Na linguagem do Lean (capítulo 3 da leitura `processos-s1.md`), os juros da dívida são desperdício: retrabalho, defeitos e espera.

📖 Ref. Martin Fowler — Is High Quality Software Worth the Cost?: https://martinfowler.com/articles/is-quality-worth-cost.html

📖 Ref. Martin Fowler — Design Stamina Hypothesis: https://martinfowler.com/bliki/DesignStaminaHypothesis.html

5. Evitar a dívida que não traz benefício

A dívida ingênua se evita com prática de engenharia. As práticas de XP, vistas na Sprint 0, existem em boa parte para isso:

Prática	Dívida que evita
Testes automatizados (TDD)	regra sem teste; receio de alterar o código
Refatoração	projeto do código que não acompanha o que a equipe aprendeu
Revisão de código e programação em par	conhecimento concentrado; solução que só o autor entende
Integração contínua	integração demorada e sujeita a erro
Definição de pronto	cartão "terminado" sem teste, sem revisão ou sem documentação

A definição de pronto merece atenção. Se "pronto" significa apenas "funciona na minha máquina", cada cartão concluído deixa um pouco de dívida para trás: um teste que não foi escrito, uma revisão que não aconteceu. Uma definição de pronto que inclui teste, revisão e CI verde impede que essa dívida entre.

O limite de WIP, da aula de Kanban, ajuda pelo mesmo caminho. Com menos trabalho aberto ao mesmo tempo, a equipe tem atenção para terminar bem o que começou, em vez de deixar vários itens "quase prontos".

Mesmo as melhores equipes criam alguma dívida, porque desenvolver software envolve incerteza. O que as diferencia é que removem dívida no mesmo ritmo em que a criam.

6. Tornar a dívida visível

Dívida que não está escrita não entra no planejamento. O primeiro passo para gerenciá-la é registrá-la onde o restante do trabalho está registrado.

Um registro útil tem cinco campos:

```
Título:     Validação de tarefa duplicada nas duas telas de cadastro
Tipo:       inevitável (surgiu quando a segunda tela foi criada)
Onde:       TelaCadastro.kt e TelaEdicao.kt
Juros:      toda regra nova precisa ser escrita duas vezes; já houve divergência
Principal:  cerca de 3 horas para extrair uma função comum e testá-la
```

O campo que mais importa é o dos juros. É ele que permite comparar uma dívida com outra, e uma dívida com uma funcionalidade nova. "Código duplicado" não ajuda a decidir; "toda regra nova é escrita duas vezes, e já divergiram" ajuda.

No GitHub, o registro é uma issue com uma etiqueta própria, no mesmo quadro do grupo:

```
gh label create divida-tecnica --description "Dívida técnica registrada" --color D93F0B
gh issue create --label divida-tecnica --title "Validação duplicada nas telas de cadastro" \
  --body "Tipo: inevitável. Onde: TelaCadastro.kt e TelaEdicao.kt. Juros: ... Principal:
        ..."
```

Com a etiqueta, o quadro pode ser filtrado para mostrar só as dívidas, e o grupo acompanha quantas estão abertas a cada sprint.

Para quem decide prioridades (o dono do produto, um cliente), a conversa funciona melhor em termos de consequências: prazo, defeitos e risco. Detalhes de código ficam para a equipe.

📖 Ref. GitHub — Managing labels: https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels

📖 Ref. GitHub CLI — gh label create: https://cli.github.com/manual/gh_label_create

7. Pagar

7.1 Quatro orientações

- Deixar o código melhor do que encontrou. Ao mexer num trecho para entregar uma funcionalidade, aproveitar para arrumá-lo. Fowler chama isso de refatoração oportunista. O efeito é que a dívida é paga onde o código mais muda, que é onde os juros são mais altos.
- Pagar primeiro a de juros altos. A prioridade vai para o código alterado com frequência e para o que causa defeitos, e não para o que mais incomoda quem lê.
- Pagar junto com trabalho de valor. Reservar parte de cada sprint para dívida rende mais do que uma "sprint de limpeza", em que nada é entregue ao usuário e a prioridade é difícil de defender.
- Pagar aos poucos. Passos pequenos, cada um com os testes passando. Uma reescrita grande, de uma vez, troca uma dívida conhecida por um risco desconhecido.

A refatoração oportunista pede um cuidado: arrumar um trecho leva a outro, e é fácil gastar o dia longe da tarefa original. A orientação é melhorar um pouco a cada passagem, sem tentar deixar tudo perfeito de uma vez.

7.2 Quando não pagar

Nem toda dívida precisa ser paga:

Situação	Por quê
Código que não será mais alterado	não cobra juros

Situação	Por quê
Protótipo descartável	será jogado fora
Produto perto do fim da vida	o investimento não retorna

Nesses casos, a decisão de não pagar é legítima, desde que seja uma decisão. Vale registrá-la, para que ninguém perca tempo reavaliando a mesma questão.

📖 Ref. Martin Fowler — Opportunistic Refactoring: https://martinfowler.com/bliki/OpportunisticRefactoring.html

8. No projeto do grupo

8.1 O exemplo do MUSI

A retrospectiva do MUSI, vista na aula anterior, encontrou dívidas que não estão no código. A situação em 29/09 era esta:

Dívida	Tipo	Juros	Pagamento previsto
Quadro parado, com três cartões em `Todo`	ingênua	ninguém sabe o estado do trabalho sem perguntar	ação A1: pôr o quadro no caminho do trabalho
`main` aceita integração sem pull request	ingênua	os dois PRs integrados não tiveram revisão	ação A2: exigir PR e CI verde (feita em parte)
A verificação local (`./verificar.sh`) depende de alguém lembrar de rodar	ingênua	falhas de CI que o script já teria apontado	ação A3: gancho `pre-push` em `.githooks/`

As ações de uma retrospectiva são um plano de pagamento: têm responsável, prazo e forma de verificar. Dez dias depois, só a ação mecânica (a proteção da `main`) tinha andado, e em parte. A lição vale para qualquer dívida: sem dono e sem data, ela tende a ficar.

8.2 Na entrega de 16/10

A dívida técnica não é um critério novo da rubrica. O levantamento feito em aula serve a duas partes da entrega da Sprint 1:

Parte da entrega	Como usar
`docs/retrospectiva-01.md`	uma dívida de juros altos pode ser um dos fatos e virar uma das ações, com responsável, prazo e verificação
Quadro no GitHub Projects	as issues com `divida-tecnica` ficam no backlog, visíveis, ao lado das histórias

Registrar uma dívida não reduz a nota. Mostra que o grupo conhece o estado do próprio projeto, que é o que a retrospectiva pede.

Na Sprint 2, a integração contínua e as métricas DORA dão números para acompanhar essas dívidas ao longo do tempo.

9. Exercícios e dúvidas frequentes

Exercícios

1. Um módulo confuso acrescenta 1 dia a cada mudança. Reorganizá-lo custa 4 dias. O grupo prevê 3 mudanças nesse módulo até o fim do semestre. Vale pagar a dívida? E se fossem 6 mudanças? A partir de quantas mudanças o pagamento compensa?
2. Classifique cada situação como dívida ingênua, inevitável ou estratégica, e diga em que quadrante de Fowler ela fica:
    - o grupo deixou os testes de uma tela para depois da apresentação, e abriu uma issue para escrevê-los na semana seguinte;
    - ninguém do grupo sabia que a biblioteca de navegação tinha rotas tipadas, e as rotas foram escritas como texto;
    - o backend escolhido mudou o formato de uma resposta, e o aplicativo deixou de ler um campo;
    - o grupo sabe que deveria revisar os PRs, mas integra direto para ganhar tempo.
3. Escolha um arquivo do projeto do grupo que foi alterado em mais de três PRs. Há alguma dívida nele? Quais são os juros?
4. Liste os PRs do grupo com `gh pr list --state merged --json number,title,files` e identifique os dois arquivos mais alterados. Eles coincidem com os trechos que o grupo considera mais difíceis de mexer?
5. Escreva o registro de uma dívida do projeto do grupo, com os cinco campos do capítulo 6, e abra a issue correspondente.
6. Releia a definição de pronto do grupo. Que tipo de dívida ela deixa passar?

Respostas do exercício 1: com 3 mudanças, os juros somam 3 dias e o principal custa 4; não compensa. Com 6 mudanças, os juros somam 6 dias; compensa. O pagamento passa a compensar a partir de 5 mudanças (com 4, os dois caminhos custam o mesmo).

Dúvidas frequentes

Todo código ruim é dívida técnica? O termo é usado assim no dia a dia, mas vale distinguir. Código ruim num trecho que ninguém altera não cobra juros. E código escrito sem cuidado, sem nenhum ganho em troca, é a dívida ingênua: a resposta a ela é melhorar a prática, e não planejar o pagamento.

Dívida técnica é o mesmo que bug? Não. Um defeito é um comportamento errado, que o usuário pode perceber. A dívida é uma característica interna, que encarece as mudanças. Defeitos conhecidos e não corrigidos, porém, entram na conta: cobram juros a cada vez que alguém esbarra neles.

Quem decide quando pagar? A equipe estima os juros e o principal; quem define as prioridades do produto decide a ordem, junto com as funcionalidades. Por isso a dívida precisa estar escrita no mesmo quadro, em termos que essa pessoa entenda.

Refatorar é o mesmo que pagar dívida? Refatorar é mudar a estrutura do código sem mudar o comportamento. É a principal forma de pagar dívida de projeto do código. Outras dívidas são pagas de outras formas: escrevendo testes, automatizando a integração, documentando uma decisão.

Uma equipe ágil deveria ter dívida zero? Não é um objetivo realista nem necessário. O que se espera é que a equipe saiba o que deve, escolha o que pagar e não deixe a dívida crescer a ponto de travar as entregas.

Como medir a dívida? Não há uma medida única. Na prática, usam-se sinais: o tempo que as mudanças levam nos trechos afetados, a quantidade de defeitos, os arquivos mais alterados e o número de issues de dívida abertas. As métricas de fluxo da Sprint 1 e as métricas DORA da Sprint 2 ajudam a acompanhar.

Referências

Todas as páginas foram consultadas em outubro de 2026.

A metáfora e o conceito

- Ward Cunningham — The WyCash Portfolio Management System (1992): https://c2.com/doc/oopsla92.html
- Ward Cunningham — Ward Explains Debt Metaphor: https://wiki.c2.com/?WardExplainsDebtMetaphor
- Martin Fowler — Technical Debt: https://martinfowler.com/bliki/TechnicalDebt.html
- Martin Fowler — Technical Debt Quadrant: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

Custo e qualidade

- Martin Fowler — Is High Quality Software Worth the Cost?: https://martinfowler.com/articles/is-quality-worth-cost.html
- Martin Fowler — Design Stamina Hypothesis: https://martinfowler.com/bliki/DesignStaminaHypothesis.html

Como pagar

- Martin Fowler — Opportunistic Refactoring: https://martinfowler.com/bliki/OpportunisticRefactoring.html
- Martin Fowler — Definition of Refactoring: https://martinfowler.com/bliki/DefinitionOfRefactoring.html
- Martin Fowler — Continuous Integration: https://martinfowler.com/articles/continuousIntegration.html

Livro

- RUBIN, Kenneth S. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley, 2012. Capítulo 8, *Technical Debt*.

Ferramentas

- GitHub — Managing labels: https://docs.github.com/en/issues/using-labels-and-milestones-to-track-work/managing-labels
- GitHub CLI — gh label create: https://cli.github.com/manual/gh_label_create